

Randomized Radial Basis Function Neural Network for Solving Multiscale Elliptic Equations

Yuhang Wu, Ziyuan Liu, Wenjun Sun, Xu Qian

1 Introduction

Multiscale problems are prevalent in scientific and engineering research, characterized by phenomena exhibiting features across multiple scales. These problems are typically modeled as partial differential equations (PDEs) with rapidly oscillating coefficients. Such equations pose significant computational challenges for traditional numerical methods, which often require exceedingly fine meshes to capture high-frequency components of the solution.

This paper focuses on the classical multiscale elliptic problem with Dirichlet boundary conditions:

$$\begin{cases} Lu^\varepsilon(x) = f(x), & x \in \Omega, \\ u^\varepsilon(x) = g(x), & x \in \partial\Omega, \end{cases} \quad (1)$$

where L is a linear operator defined as $Lu^\varepsilon(x) = -\operatorname{div}(A^\varepsilon(x)\nabla u^\varepsilon(x))$. Here, Ω is a bounded subset of $\mathbb{R}^n$ and $\varepsilon \ll 1$ represents the ratio of the smallest scale to the largest scale in the physical problem. The coefficient matrix $A^\varepsilon(x) = (a_{ij}^\varepsilon(x))$ is assumed to be symmetric and positive definite.

While various methods have been proposed to solve multiscale elliptic problems, including homogenization, multiscale finite element methods (MsFEM), and domain decomposition, these approaches often struggle with computational cost and accuracy when dealing with complex geometries and high frequencies. Neural network-based methods have emerged as promising alternatives, avoiding the complex mesh generation process. However, conventional deep neural networks (DNNs) face challenges when addressing multiscale and high-frequency PDEs due to the Frequency Principle (F-Principle), which indicates that DNNs tend to learn low-frequency components better than high-frequency ones.

2 Randomized Radial Basis Function Neural Network

2.1 Radial Basis Function Neural Network

The radial basis function neural network (RBFNN) is a specialized three-layer artificial neural network consisting of an input layer, a hidden layer, and an output layer. The hidden layer uses a radial basis function (RBF) as its activation function, which is a real-valued function whose value depends only on the distance from a center point:

$$\rho(x) = \rho(\|x - c\|), \quad (2)$$

where $\|\cdot\|$ typically denotes the standard Euclidean distance.

This paper employs the Gaussian function as the RBF:

$$\rho(x, c, d) = e^{-\|d(x-c)\|^2}, \quad (3)$$

where $x = (x_1, \dots, x_n)^T$ is the input variable, $d = \text{diag}(d_1, \dots, d_n)$ controls the radial range (width coefficient), and $c = (c_1, \dots, c_n)^T$ is the center coefficient.

The output of the RBFNN is given by:

$$u_N(x) = \sum_{i=1}^J w_i \rho(x, c_i, d_i) = \sum_{i=1}^J w_i e^{-\|d_i(x-c_i)\|^2}, \quad (4)$$

where J is the number of neurons in the hidden layer, and w_i are the weights connecting the hidden layer to the output.

For simplicity, the width coefficient is often assumed to be uniform across dimensions, leading to:

$$u_N(x) = \sum_{i=1}^J w_i e^{-\sigma_i \|x-c_i\|^2}, \quad (5)$$

where σ_i is called the shape coefficient of the i -th Gaussian function.

RBFNNs possess excellent local approximation properties, making them particularly well-suited for capturing the localized features of multiscale problems. Additionally, RBFNNs have been proven to have the best approximation property, an advantage over back-propagation networks.

2.2 Randomized RBFNN Framework

Traditional RBFNN training often requires specialized algorithms to select optimal width, center, and weight coefficients, which can still be computationally expensive. Drawing inspiration from Extreme Learning Machines (ELMs), this paper introduces the Randomized RBFNN (RRNN) method, which assigns random parameters to the RBF kernels.

In RRNN, the output function with J hidden neurons is represented by:

$$u_N(x) = \sum_{i=1}^J w_i \rho(c_i, \sigma_i, x), \quad x \in \mathbb{R}^n, w_i \in \mathbb{R}^m, c_i \in \mathbb{R}^n, \sigma_i \in \mathbb{R}^+, \quad (6)$$

where $\rho(c_i, \sigma_i, x)$ denotes the activation function of the i -th hidden neuron.

For Q arbitrary distinct samples $(x_j, u_j) \in \mathbb{R}^n \times \mathbb{R}^m$, RRNN with J hidden neurons aims to model:

$$\sum_{i=1}^J w_i \rho(c_i, \sigma_i, x_j) = u_j, \quad j = 1, \dots, Q, \quad (7)$$

which can be written in matrix form as:

$$Aw = b, \quad (8)$$

where

$$A = \begin{pmatrix} \rho(c_1, \sigma_1, x_1) & \cdots & \rho(c_J, \sigma_J, x_1) \\ \vdots & \ddots & \vdots \\ \rho(c_1, \sigma_1, x_Q) & \cdots & \rho(c_J, \sigma_J, x_Q) \end{pmatrix}_{Q \times J}, \quad w = \begin{pmatrix} w_1^T \\ \vdots \\ w_J^T \end{pmatrix}_{J \times m}, \quad b = \begin{pmatrix} u_1^T \\ \vdots \\ u_Q^T \end{pmatrix}_{Q \times m}. \quad (9)$$

A key theoretical foundation for the RRNN approach is provided by the following theorem:

Theorem 1. *Given any small positive value $\epsilon > 0$, an activation function $\rho : \mathbb{R}^n \rightarrow \mathbb{R}$ which is infinitely differentiable in any interval, and Q arbitrary distinct samples $(x_i, u_i) \in \mathbb{R}^n \times \mathbb{R}^m$, there exists $J \leq Q$ such that for any $\{c_i, \sigma_i\}_{i=1}^J$ randomly generated from any intervals of $\mathbb{R}^n \times \mathbb{R}$ according to any continuous probability distribution, with probability one, $\|Aw - b\| \leq \epsilon$.*

Based on this theorem, the parameters for the hidden layer can be pre-set to uniform random values:

$$\sigma_i \in U([0, \beta]), \quad c_i \in U([-1, 1]^n), \quad i = 1, \dots, J, \quad (10)$$

where β is a positive constant representing the upper bound of the random shape coefficient σ .

Once these coefficients are randomized, they remain fixed throughout the training process. The output then becomes a linear combination of basis functions $\{\rho_i(x)\}_{i=1}^J$, defining a randomized radial basis function neural network space:

$$U_R(\Omega) = \left\{ u_N(w, x) = \sum_{i=1}^J w_i \rho_i(x) : x \in \Omega \right\} = \text{span}\{\rho_1(x), \rho_2(x), \dots, \rho_J(x)\}. \quad (11)$$

3 RRNN Method for Multiscale Elliptic Equations

3.1 Challenges of Multiscale Elliptic Equations

Multiscale elliptic equations present several challenges:

1. The coefficient $A^\varepsilon(x)$ exhibits violent oscillatory characteristics with strong high-frequency components. 2. The derivative of $A^\varepsilon(x)$ also oscillates violently with large differences in magnitude. 3. The high-frequency nature of $A^\varepsilon(x)$ leads to solutions and their derivatives with strong localized features.

These challenges require computational methods capable of capturing localized high-frequency information efficiently and accurately.

3.2 Domain Decomposition and Normalization

To address these challenges, the RRNN method employs domain decomposition, dividing the computational domain Ω into multiple non-overlapping subdomains. This approach transforms high-frequency data into a low-frequency space within each subdomain, making it easier to capture local solution properties using RBFNNs.

3.3 Weak Formulation and Discretization

For multiscale elliptic equations, the RRNN method uses a weak formulation to transfer the derivatives of $A^\varepsilon(x)$ and $\nabla u^\varepsilon(x)$ to the derivatives of a test function $v(x)$, avoiding difficulties related to drastic gradient changes.

The weak formulation of the problem is: Find $u \in H_g^1(\Omega)$ such that

$$\int_{\Omega} (A^\varepsilon \nabla u) \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in H_0^1(\Omega), \quad (12)$$

where $H_g^1(\Omega) = \{u \in H^1(\Omega) : u = g \text{ on } \partial\Omega\}$.

In the RRNN method, the domain is partitioned into S non-overlapping subdomains. Let $\{T_h\}$ be this partition of Ω , and let Γ be the union of boundaries of all subdomains. The trial function space in each subdomain K is chosen as the randomized radial basis function space $U_R(K)$, while the test function space $V_h(K)$ is chosen as a finite-dimensional space.

The RRNN method for solving the equations is formulated as: Find $u_\rho \in U = \{u \in L^2(\Omega) : u|_K \in U_R(K), \forall K \in T_h\}$ such that

$$\int_K (A^\varepsilon \nabla u_\rho) \cdot \nabla v_k \, dx - \int_{\partial K} (A^\varepsilon \nabla u_\rho) \cdot n_K v_k \, ds = \int_K f v_k \, dx \quad \forall v_k \in V_h, \forall K \in T_h, \quad (13)$$

$$n^+ u_\rho(x_j^c)|_{\partial K^+} + n^- u_\rho(x_j^c)|_{\partial K^-} = 0 \quad \forall x_j^c \in C, \quad (14)$$

$$n^+ \cdot \nabla u_\rho(x_j^c)|_{\partial K^+} + n^- \cdot \nabla u_\rho(x_j^c)|_{\partial K^-} = 0 \quad \forall x_j^c \in C, \quad (15)$$

$$u_\rho(x_j^b) = g(x_j^b) \quad \forall x_j^b \in B, \quad (16)$$

where B is the set of collocation points on the domain boundary, C is the set of collocation points on the interior boundary, and n_K is the unit outer normal vector on ∂K .

The test function space V_h is composed of localized test functions defined as:

$$v_k = \begin{cases} \bar{v} \neq 0, & \text{over } K, \\ 0, & \text{over } K^c, \end{cases} \quad K \cup K^c = \Omega. \quad (17)$$

In one-dimensional problems, the non-vanishing test function $\bar{v}(x)$ is chosen to be high-order polynomials $P_{k+1}(x) - P_{k-1}(x)$, where $P_k(x)$ is the Legendre polynomial of order k . In two-dimensional problems, using the tensor product rule, $\bar{v}(x, y)$ can be chosen as $(P_{k_1+1}(x) - P_{k_1-1}(x))(P_{k_2+1}(y) - P_{k_2-1}(y))$.

3.4 Linear System and Solution

From the equations, the component matrices A and source term vector b of the final linear system can be defined as:

$$\begin{pmatrix} A_e \\ A_b \\ A_c \end{pmatrix} w = \begin{pmatrix} b_e \\ b_b \\ b_c \end{pmatrix}, \quad (18)$$

where A_e is a $SQ \times SJ$ matrix, b_e is a $SQ \times 1$ vector, A_b is a $N_b \times SJ$ matrix, b_b is a $N_b \times 1$ vector, A_c is a $N_c \times SJ$ matrix, b_c is a $N_c \times 1$ vector, and w is a $SJ \times 1$ vector of unknown weight variables.

The best value of w is found using the least squares method:

$$\hat{w} = \arg \min \|Aw - b\|. \quad (19)$$

3.5 Algorithm

The complete RRNN algorithm for solving multiscale elliptic equations is as follows:

1. Initialize the radial basis function network architecture.
2. Randomly choose parameters σ and c from a uniform distribution and fix them.
3. Decompose the domain into S subdomains and choose Q test functions $v_k \in V_h$ in each subdomain.
4. Assemble the linear system $A_e w = b_e$ based on the weak formulation.
5. Randomly select points $\{x_j^b\}_{j=1}^{N_b}$ and assemble the linear system $A_b w = b_b$.
6. Randomly select points $\{x_j^c\}_{j=1}^{N_c}$ and assemble the linear system $A_c w = b_c$.
7. Solve the linear system by the linear least-squares method to obtain w .
8. Compute the solution $u(x_{test})$ using the trained network.

4 Numerical Examples and Results

The performance of the RRNN method was evaluated through various numerical examples, including one-dimensional and two-dimensional problems, two-scale and three-scale problems, and Poisson-Boltzmann problems. The method was compared with several neural network approaches, including those based on gradient descent (PINN, DRM, MscaleDNN, SRBFNN) and those based on least squares (LocELM).

Computational accuracy was measured using relative maximum error and relative root mean square error:

$$\max \text{ error} = \frac{\|u_N - u\|_{L^\infty}}{\|u\|_{L^\infty}}, \quad \text{rms error} = \frac{\|u_N - u\|_{L^2}}{\|u\|_{L^2}}, \quad (20)$$

where u_N is the approximate solution from the RRNN method and u is the reference solution.

4.1 One-Dimensional Examples

4.1.1 Periodic Coefficient Case

The first example considered a periodic coefficient:

$$A^\varepsilon(x) = (2 + \cos(2\pi x/\varepsilon))^{-1}, \quad (21)$$

with exact solution:

$$u(x) = x - x^2 + \varepsilon \left(\frac{1}{4\pi} \sin(2\pi x/\varepsilon) - \frac{1}{2\pi} x \sin(2\pi x/\varepsilon) \right) - \varepsilon^2 \left(\frac{1}{4\pi^2} \cos(2\pi x/\varepsilon) + \frac{1}{4\pi^2} \right), \quad (22)$$

and boundary conditions $u(0) = u(1) = 0$ with source term $f(x) = 1$.

The RRNN method achieved remarkable accuracy, with errors on the order of 10^{-13} for $\varepsilon = 0.5$ and 10^{-10} for $\varepsilon = 0.005$. The training times were all under 10 seconds, indicating excellent computational efficiency.

4.1.2 Double-Scale Case

The second example involved a double-scale coefficient:

$$A^\varepsilon(x) = 2 + \sin(2\pi x/\varepsilon) \cos(2\pi x), \quad (23)$$

with $f(x) = 1$ and $g(x) = 1$.

Compared to methods based on gradient descent (SRBFNN, MscaleDNN, DRM, PINN), the RRNN method achieved 2-3 orders of magnitude reduction in error and significantly faster training times (seconds vs. hundreds or thousands of seconds). When compared to LocELM, which also uses domain decomposition and least squares, RRNN consistently achieved higher accuracy with equivalent computational cost, particularly for smaller values of ε .

4.1.3 Three-Scale Problem

The third example considered a three-scale coefficient:

$$A^\varepsilon(x) = (2 + \cos(2\pi x/\varepsilon_1))(2 + \cos(2\pi x/\varepsilon_2)), \quad (24)$$

with $f(x) = 1$ and $g(x) = 1$.

This more complex case presented additional challenges due to increased oscillations and localized features. The RRNN method still achieved errors within the order of 10^{-5} to 10^{-6} and maintained its computational efficiency advantage over other methods.

4.2 Two-Dimensional Examples

4.2.1 Smooth Source Function Case

The first two-dimensional example considered:

$$A^\varepsilon(x, y) = \frac{1}{4 + \cos(2\pi(x^2 + y^2)/\varepsilon)}, \quad (25)$$

with smooth source function $f(x, y) = -(x^2 + y^2)$ and exact solution:

$$u(x, y) = \frac{1}{4}(x^2 + y^2)^2 + \frac{\varepsilon}{16\pi}(x^2 + y^2) \sin(2\pi(x^2 + y^2)/\varepsilon) + \frac{\varepsilon^2}{32\pi^2} \cos(2\pi(x^2 + y^2)/\varepsilon). \quad (26)$$

The RRNN method achieved absolute point-wise errors within the orders of 10^{-8} , 10^{-6} , and 10^{-4} for $\varepsilon = 0.5, 0.2, 0.1$, respectively. When compared to LocELM with equivalent computational cost, RRNN consistently demonstrated superior accuracy.

4.2.2 Double-Scale Coefficient Case

The second two-dimensional example involved a coefficient with double-scale:

$$A^\varepsilon(x, y) = \frac{1.5 + \sin(2\pi x/\varepsilon)}{1.5 + \sin(2\pi y/\varepsilon)} + \frac{1.5 + \sin(2\pi y/\varepsilon)}{1.5 + \cos(2\pi x/\varepsilon)} + \sin(4x^2 y^2) + 1, \quad (27)$$

with $f(x) = -10$ and $g(x) = 0$.

This example featured two different frequency components and violent oscillations, presenting challenges for conventional methods. The RRNN method outperformed both gradient descent-based methods and LocELM, achieving 1-2 orders of magnitude lower errors and significantly faster training times.

4.2.3 Poisson-Boltzmann Equation

The final example considered the Poisson-Boltzmann equation:

$$\begin{cases} -\operatorname{div}(A^\varepsilon(x, y)\nabla u^\varepsilon(x, y)) + \kappa(x, y)u^\varepsilon(x, y) = f(x, y), & (x, y) \in \Omega, \\ u^\varepsilon(x, y) = g(x, y), & (x, y) \in \partial\Omega, \end{cases} \quad (28)$$

where $\kappa(x, y) = \pi^2$ and $A^\varepsilon(x, y) = 1 + 0.5 \cos(10\pi x) \cos(20\pi y)$.

The exact solution was set as $u(x, y) = \sin(\pi x) \sin(\pi y) + 0.05 \sin(10\pi x) \sin(20\pi y)$, determining the corresponding boundary conditions and source term.

The RRNN method achieved absolute point-wise errors predominantly on the order of 10^{-4} across most regions. Compared to other methods, RRNN offered accuracy four orders of magnitude higher than PINN and two orders of magnitude higher than other methods, while maintaining significantly shorter training times.

5 Implications and Key Findings

5.1 Parameter Influence

The performance of the RRNN method is influenced by several key parameters:

1. Number of Neurons: Increasing the number of neurons per subdomain initially yields an exponential reduction in error, which plateaus beyond a certain threshold.
2. Number of Test Functions: A similar pattern is observed with test functions, where an optimal number exists beyond which marginal improvements or even slight degradations might occur.
3. Number of Subdomains: The accuracy generally improves with more subdomains, with optimal results typically achieved when the number of subdomains equals $1/\varepsilon$.
4. Upper Bound of Random Shape Coefficient: The error tends to deteriorate when this parameter is either substantially large or small, while the training time remains largely unaffected.

5.2 Advantages over Gradient Descent Methods

The RRNN method demonstrates several advantages over traditional gradient descent-based neural network methods:

1. Accuracy: RRNN consistently achieves 2-3 orders of magnitude lower errors across various problems and scale parameters.
2. Efficiency: Training times are reduced by 1-2 orders of magnitude compared to gradient descent methods.
3. Robustness: The method maintains high accuracy even as the scale parameter ε decreases to very small values.

5.3 Advantages over Other Least Squares Methods

When compared to LocELM, which also uses domain decomposition and least squares:

1. Higher Accuracy: RRNN achieves better results with equivalent computational cost, particularly for smaller values of ε .
2. Better High-Frequency Capture: The radial basis function approach allows RRNN to better capture localized high-frequency features of the solution.
3. Consistent Performance: RRNN maintains optimal performance with a fixed number of collocation points, whereas LocELM requires an increasing number to achieve comparable accuracy as ε decreases.

5.4 Future Research Directions

While the RRNN method shows promising results, several aspects warrant further investigation:

1. Efficient Resolution for Smaller Scale Ratios: As ε decreases, more subdomains are required, leading to larger matrices and extended training times. Alternative approaches might be developed to mitigate this challenge.
2. Theoretical Error Analysis: Establishing a theoretical framework analogous to finite element analysis would help analyze the error associated with the RRNN method.
3. Extension to Nonlinear Problems: The current method focuses on linear elliptic equations, but its extension to nonlinear problems could broaden its applicability.